Progressive Electricity Billing System

1. Problem Statement

Develop a Python program to calculate household electricity bills based on electricity consumption units. The system uses multiple consumption slabs, fixed charges, surcharges, and rebates. It also compares electricity bills across several months and identifies unusual increases in consumption or billing amounts.

2. Computational Thinking

Computational thinking is the process of solving a problem step by step using logical methods.

1. Decomposition

- Get the number of months.
- Enter electricity units for each month.
- Calculate charges based on slabs.
- Add fixed charges and surcharges.
- Apply rebates.
- Compare monthly bills.
- Identify unusual increases.

2. Pattern Recognition

- Higher electricity consumption results in higher bills.
- Fixed charges are added every month.
- Rebates are given for low consumption.
- Unusual increases can be identified by comparing consecutive months.

3. Abstraction

- Consider only units consumed, slab rates, fixed charges, surcharge, and rebate.
- Ignore unnecessary household details.

4. Algorithm Design

- Create steps to calculate and compare electricity bills.

3. Algorithm

1. Start.

2. Read the number of months.
3. Create empty lists for units and bills.
4. Repeat for each month:
    - Read electricity units.
    - Calculate charges using slabs:
        - First 100 units = ₹2 per unit.
        - Next 100 units = ₹3 per unit.
        - Above 200 units = ₹5 per unit.
    - Add fixed charge of ₹50.
    - Calculate 5% surcharge.
    - If units are below 50, give ₹20 rebate.
    - Calculate final bill.
    - Store units and bill in lists.
5. Compare consecutive monthly bills.
6. If the current bill increases by more than 20%, display an unusual increase warning.
7. Display all monthly bills.
8. Stop.

4. Variables and Data Types

Variable	**Data type**	**Purpose**
months	Integer	Number of months
units	Integer	Electricity consumption
bill	Float	Final electricity bill
charge	Float	Slab charges
fixed	Integer	Fixed charge
surcharge	Float	Extra charge
rebate	Integer	Discount

bills	List	Stores monthly bills
readings	List	Stores monthly units
month_name	String	Month description
record	Tuple	Stores month and bill

5. Operators Used

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
>	Greater than
<	Less than
==	Equal to
+=	Increment
and	Logical AND

6. Conditional Statements

- if: Checks whether units are below 100.
- elif: Checks whether units are between 100 and 200.
- else: Calculates charges for units above 200.
- Another if checks whether units are below 50 to apply the rebate.
- Another if checks whether the bill increased by more than 20%.

7. Loops

- for loop: Repeats bill calculation for each month.
- for loop: Displays monthly bills.
- if condition inside the loop: Detects unusual increases.

8. Strings and Pattern Generation

Strings: Used to display month names and billing messages.

Pattern generation: Displays a simple monthly bill report.

ELECTRICITY BILL

Month Units Bill

January 80 218.40

February 120 407.00

March 250 945.00

---

9. Lists

Lists store multiple months' electricity readings and bills.

readings = [80, 120, 250]bills = [218.40, 407.00, 945.00]

10. Tuples and Tuple Operations

Tuples store fixed monthly records.

record = ("January", 80, 218.40)print(record[0])print(record[1])print(record[2])

Tuple unpacking:

month, units, bill = recordprint(month, units, bill)

11. Pseudocode

START

INPUT number_of_months

CREATE readings list

CREATE bills list

FOR each month:

```
    INPUT units

    IF units <= 100:
        charge = units * 2

    ELSE IF units <= 200:
        charge = 100 * 2 + (units - 100) * 3

    ELSE:
        charge = 100 * 2 + 100 * 3
                + (units - 200) * 5

    fixed = 50
    surcharge = (charge + fixed) * 0.05

    IF units < 50:
        rebate = 20
    ELSE:
        rebate = 0

    bill = charge + fixed + surcharge - rebate

    ADD units TO readings
    ADD bill TO bills

    DISPLAY bill

FOR each month after first month:
```

increase = bills[current] - bills[previous]

IF increase > bills[previous] * 0.20:

DISPLAY "Unusual increase detected"

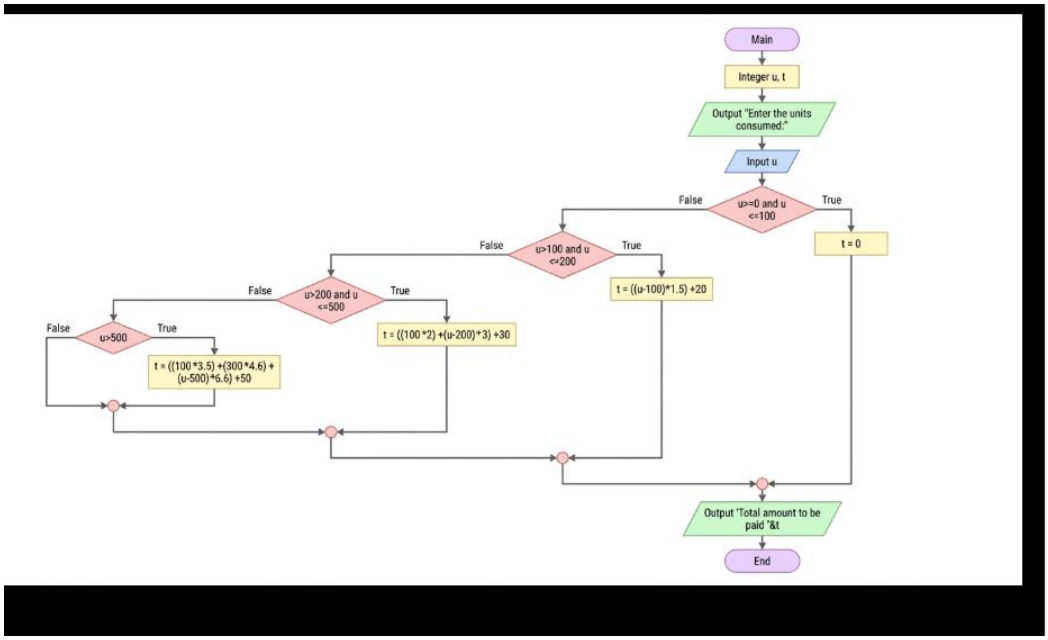
DISPLAY all monthly bills

STOP

12. Python Program

months = int(input("Enter number of months: "))readings = []bills = []for i in range(months): print("\nMonth", i + 1) units = int(input("Enter electricity units: ")) if units <= 100: charge = units * 2 elif units <= 200: charge = 100 * 2 + (units - 100) * 3 else: charge = 100 * 2 + 100 * 3 + (units - 200) * 5 fixed = 50 surcharge = (charge + fixed) * 0.05 if units < 50: rebate = 20 else: rebate = 0 bill = charge + fixed + surcharge - rebate readings.append(units) bills.append(bill) print("Electricity Bill: Rs.", round(bill, 2))print("\n------ MONTHLY BILL REPORT ------")for i in range(months): record = ("Month " + str(i + 1), readings[i], bills[i])

13. Flowchart:



Flowchart steps:

1. Start.
2. Input number of months.
3. Input electricity units.
4. Calculate slab charges.
5. Add fixed charges.
6. Calculate surcharge.
7. Check rebate eligibility.
8. Calculate final bill.
9. Store monthly bill.
10. Check whether all months are completed.
11. Compare bills and identify unusual increases.
12. Display report.
13. Stop.

14. Sample Output

Enter number of months: 3

Month 1

Enter electricity units: 80

Electricity Bill: Rs. 218.5

Month 2

Enter electricity units: 120

Electricity Bill: Rs. 365.75

Month 3

Enter electricity units: 250

Electricity Bill: Rs. 1039.25

------ MONTHLY BILL REPORT ------

Month 1 Units: 80 Bill: 218.5

Month 2 Units: 120 Bill: 365.75

Month 3 Units: 250 Bill: 1039.25

Warning: Unusual bill increase!

Total Bill: 1623.5

Average Bill: 541.17

15. Conclusion

The Progressive Electricity Billing System calculates monthly electricity bills using different consumption slabs, fixed charges, surcharges, and rebates. It uses computational thinking, algorithms, conditional statements, loops, strings, lists, tuples, and tuple unpacking. It also compares monthly bills and detects unusual increases, helping households monitor electricity usage.